

Capítulo 11 — Taller ESP32: PWM, ADC e I2C

Objetivo del capítulo

Programar el ESP32 en Arduino IDE para: control PWM con LEDC, leer ADC con divisor de voltaje, comunicarnos por I2C, y construir proyectos prácticos (dimmer de LED, monitor de batería).

11.1 LEDC — Control PWM Avanzado

El ESP32 tiene 16 canales LEDC (LED Control) independientes, a diferencia del `analogWrite()` limitado de Arduino. Los canales se dividen en 8 de alta velocidad y 8 de baja velocidad.

Arquitectura LEDC:

Característica	Valor
Canales disponibles	16 (GPIO 0-33, excepto 20, 24, 28-31)
Resolución configurable	1 a 16 bits
Frecuencia máxima	40 MHz (1 bit res)
Frecuencia típica LED	1 kHz - 5 kHz
Frecuencia para dimmer	>200 Hz (evitar parpadeo visible)
Timer independientes	4 (cada canal asociado a uno)

Código: Configuración básica LEDC

```
// ESP32 LEDC PWM — Ejemplo básico
// Canal 0, GPIO 2, 1kHz, resolución 13 bits

#define LED_PIN    2    // GPIO2 (LED integrado en muchas placas)
#define PWM_CH     0
#define PWM_FREQ   1000 // 1 kHz
#define PWM_RES    13   // 13 bits = 8192 niveles

void setup() {
  // Método nuevo ESP-Arduino Core 3.x:
  ledcAttach(LED_PIN, PWM_FREQ, PWM_RES);

  // Método legacy (ESP-Arduino Core 2.x):
```

```
// ledcSetup(PWM_CH, PWM_FREQ, PWM_RES);
// ledcAttachPin(LED_PIN, PWM_CH);
}

void loop() {
  // Fade de 0 a máximo y vuelta
  for (int duty = 0; duty <= 8191; duty += 50) {
    ledcWrite(LED_PIN, duty);
    delay(5);
  }
  for (int duty = 8191; duty >= 0; duty -= 50) {
    ledcWrite(LED_PIN, duty);
    delay(5);
  }
}
```

🔍 Nota sobre ledcWrite

En ESP-Arduino Core 3.x, `ledcWrite(pin, duty)` toma el GPIO directamente. En versión 2.x toma el canal. Para portabilidad, usa el nuevo API con el pin como primer argumento.

Resolución vs. Frecuencia: El producto $\text{Frecuencia} \times 2^{\text{Resolución}}$ debe ser ≤ 80 MHz (timer clock).

Ejemplo:

13 bits @ 1 kHz $\rightarrow 8192 \times 1000 = 8.2$ MHz ✓

10 bits @ 10 kHz $\rightarrow 1024 \times 10000 = 10.2$ MHz ✓

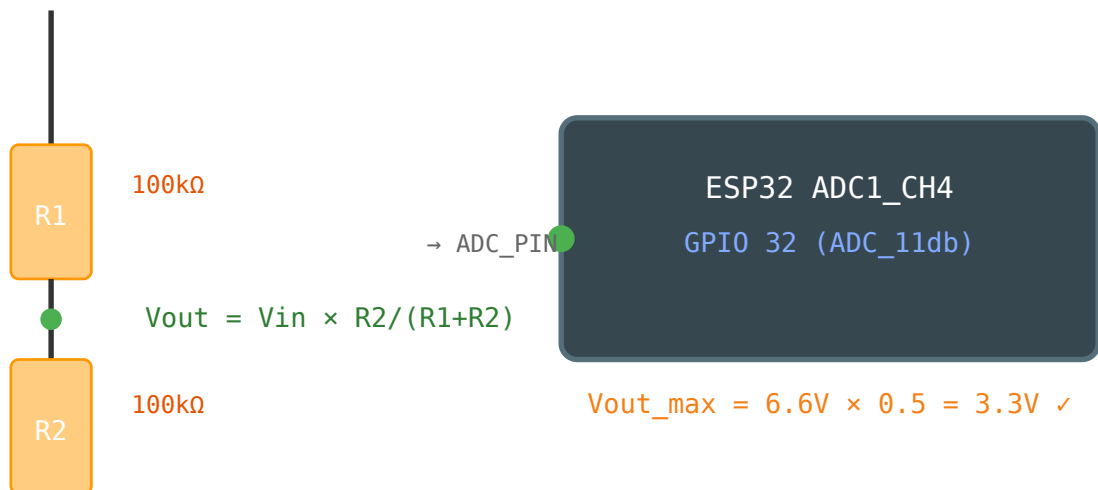
12 bits @ 20 kHz $\rightarrow 4096 \times 20000 = 81.9$ MHz ✗ (reducir resolución)

11.2 ADC — Lectura con Divisor de Voltaje

El ESP32 tiene dos ADC de 12 bits (ADC1: GPIO 32-39, ADC2: GPIO 0,2,4,12-15,25-27). La tensión de referencia por defecto es 0-3.3V, aunque se puede atenuar para rangos mayores.

Atenuación configurables:

Atenuación	Rango de entrada	Aplicación
ADC_0db	0 - 1.1V	Referencia interna calibrada
ADC_2_5db	0 - 1.5V	Sensores 1.2V
ADC_6db	0 - 2.2V	Divisores resistivos 3.3V
ADC_11db	0 - 3.3V	Rango completo por defecto

V_{in} (0-6.6V)

Divisor resistivo para medir hasta 6.6V con ADC del ESP32

Código: Lectura ADC con promediado

```
// ESP32 ADC con divisor de voltaje y promediado
#define ADC_PIN    32
#define R1          100000.0 // 100kΩ
#span class="pin">R2          100000.0 // 100kΩ
#define SAMPLES     32
#define VREF         3.3

void setup() {
  Serial.begin(115200);
  analogSetAttenuation(ADC_11db); // Rango completo 0-3.3V
}

void loop() {
  float suma = 0;
  for (int i = 0; i < SAMPLES; i++) {
    suma += analogRead(ADC_PIN);
  }
  float raw = suma / SAMPLES;

  float v_adc = (raw / 4095.0) * VREF;
  float vin = v_adc * ((R1 + R2) / R2);

  Serial.printf("Raw: %.0f | V_ADC: %.2fV | Vin: %.2fV\n", raw, v_adc, pin);
  delay(500);
}
```

⚠ Limitaciones del ADC ESP32

El ADC del ESP32 tiene notable no-linealidad (DNL/INL) y ruido. Para mediciones de precisión:

- Usa promediado (oversampling) — 32-64 muestras reduce ruido 2-3 bits efectivos
- Calibra con voltaje conocido si necesitas precisión >1%
- Considera un ADC externo (ADS1115 por I2C) para precisiones de 16 bits

11.3 Comunicación I2C

I2C (Inter-IC Bus) es un bus serial síncrono multi-master/multi-slave que usa: SDA (datos) y SCL (reloj). El ESP32 tiene 2 controladores I2C.

Parámetros I2C ESP32:

Modo	Velocidad	Notas
Standard Mode	100 kbps	Más compatible, funciona con todos los dispositivos
Fast Mode	400 kbps	Estándar para la mayoría de sensores
Fast Mode Plus	1 Mbps	Algunos dispositivos modernos
Clock stretching	Sí (configurable)	Slave puede ralentizar al master

>

Código: Escaneo de bus I2C

```
// Escáner I2C para ESP32
#include <Wire.h>

#define SDA_PIN 21
#define SCL_PIN 22

void setup() {
  Serial.begin(115200);
  Wire.begin(SDA_PIN, SCL_PIN);
  Wire.setClock(100000); // 100 kHz

  Serial.println("Escaneando bus I2C...");
  for (byte addr = 1; addr < 127; addr++) {
    Wire.beginTransmission(addr);
    byte error = Wire.endTransmission();
    if (error == 0) {
      Serial.printf("Dispositivo encontrado: 0x%02X\n", addr);
    }
  }
}
```

```
void loop() {}
```

Código: Leer sensor I2C (ejemplo BMP280)

```
// Leer dirección de un sensor I2C genérico
void leerSensorI2C(uint8_t addr, uint8_t reg) {
  Wire.beginTransmission(addr);
  Wire.write(reg);           // Registro a leer
  Wire.endTransmission(false); // Repeated start
  Wire.requestFrom(addr, (size_t)2, true);

  if (Wire.available() >= 2) {
    uint8_t high = Wire.read();
    uint8_t low  = Wire.read();
    int16_t valor = (high << 8) | low;
    Serial.printf("Valor: %d\n", valor);
  }
}

// Uso típico lectura multi-registro:
leerSensorI2C(0x76, 0xF7); // BMP280: registro presión MSB
```

💡 Pull-ups I2C obligatorios

I2C requiere resistencias pull-up en SDA y SCL. Para 100kHz y bus corto (<30cm):

- **4.7 kΩ** para 3.3V (estándar)
- **2.2 kΩ** para buses largos o Fast Mode (400kHz)

Muchos módulos ya incluyen pull-ups. Si conectas varios módulos, los pull-ups se paralelizan y pueden ser demasiado fuertes → usa 10kΩ en ese caso.

11.4 Proyecto 1 — Dimmer de LED con PWM

Controlar el brillo de un LED blanco de 3W con potenciómetro. El LED se conecta a través de un MOSFET de nivel lógico (AO3400 o IRLZ44N) manejado por PWM del ESP32.

Conexiones del circuito:

Componente	Valor	Conexión
LED 3W (cátodo)	Forward $\approx$ 3.2V	Drain MOSFET
Resistencia R_limit	0.5Ω (para limitar corriente)	Source a GND
MOSFET (AO3400)	Rds(on) = 24mΩ @ 2.5V	Gate → GPIO 25 (PWM)
Potenciómetro	100kΩ	Wiper → GPIO 34 (ADC)

Componente	Valor	Conexión
Fuente 5V	2A	LED ánodo a través de R

```
// =====
// PROYECTO: Dimmer LED con Potenciómetro
// ESP32 - ADC lee pot, PWM controla LED
// =====

#define PWM_LED    25    // Gate MOSFET
#define ADC_POT    34    // Potenciómetro
#define PWM_FREQ   2000 // 2kHz - sin parpadeo
#define PWM_RES    12    // 12 bits = 4096 niveles

void setup() {
  Serial.begin(115200);
  ledcAttach(PWM_LED, PWM_FREQ, PWM_RES);
  analogSetAttenuation(ADC_11db);
}

void loop() {
  int pot = analogRead(ADC_POT);    // 0 - 4095
  int pot = map(pot, 0, 4095, 0, 4095);

  // Opcional: aplicar corrección gamma para percepción lineal
  float gamma = pot / 4095.0;
  gamma = pow(gamma, 2.2);
  int duty = (int)(gamma * 4095);

  ledcWrite(PWM_LED, duty);

  Serial.printf("Pot: %d → Duty: %d (0.1f%%)\n", pot, duty, (duty*100.0/4095));
  delay(20);
}
```

⚠ Corrección Gamma

La luminosidad percibida sigue una potencia ~ 2.2 . Sin corrección gamma, el LED parece "volar" en los niveles bajos. Aplicar `pow(valor_normalizado, 2.2)` antes de asignar duty.

11.5 Proyecto 2 — Monitor de Batería Li-Ion

Monitorear la carga de una batería 18650 (3.7V nominal, rango operativo 3.0V - 4.2V). Usamos divisor resistivo para llevar el voltaje al rango del ADC.

Cálculo del divisor:

Vin máximo = 4.2V

Queremos Vout_max = 3.3V (rango ADC_11db)

Relación: $R2/(R1+R2) = 3.3/4.2 = 0.786$

$R1 = 22k\Omega$, $R2 = 75k\Omega \rightarrow \text{ratio} = 75/(22+75) = 0.773 \checkmark$

Corriente divisor: $4.2 / (22k + 75k) = 43 \mu A$ (acceptable)

```
// =====
// PROYECTO: Monitor Batería Li-Ion 18650
// Con divisor 22kΩ + 75kΩ en GPIO 35
// =====

#define ADC_BAT    35    // ADC1_CH7
#define R1         22000.0
#define R2         75000.0
#define SAMPLES    64
#define VREF        3.3

// Curva descarga Li-Ion simplificada (voltaje → %)
// Mapeo aproximado para descarga 0.5C:
// 4.20V = 100% | 3.90V = 70% | 3.60V = 35% | 3.30V = 10% | 3.00V = 0%

void setup() {
    Serial.begin(115200);
    analogSetAttenuation(ADC_11db);
}

void loop() {
    double suma = 0;
    for (int i = 0; i < SAMPLES; i++) {
        suma += analogRead(ADC_BAT);
    }
    float raw = suma / SAMPLES;
    double v_adc = (raw / 4095.0) * VREF;
    double v_bat = v_adc * ((R1 + R2) / R2);

    // Calcular porcentaje de carga
    int porc = calcularSoC(v_bat);

    Serial.printf("Vbatería: %.2fV | Carga: %d%% | Raw: %.0f\n", v_bat, porc, raw);

    // Alerta baja batería
    if (v_bat < 3.1) {
        Serial.println("⚠ BATERÍA BAJA — Desconectar carga");
    }

    delay(1000);
}

int calcularSoC(double voltaje) {
    if (voltaje >= 4.20) return 100;
    if (voltaje >= 4.00) return 80 + ((voltaje - 4.00) / 0.20) * 20;
    if (voltaje >= 3.80) return 40 + ((voltaje - 3.80) / 0.20) * 40;
    if (voltaje >= 3.60) return 15 + ((voltaje - 3.60) / 0.20) * 25;
    if (voltaje >= 3.00) return ((voltaje - 3.00) / 0.60) * 15;
}
```

```
return 0;
}
```

11.6 Proyecto Integrador — Sensor I2C + Display + PWM

Unión de todo lo aprendido: leer temperatura/humedad de un sensor I2C (AHT20 o SHT30), mostrar en OLED SSD1306, y controlar un ventilador con PWM según la temperatura.

Componentes necesarios:

AHT20 (I2C addr 0x38) — sensor temperatura/humedad

SSD1306 OLED 128×64 (I2C addr 0x3C)

Ventilador 5V → MOSFET controlado por GPIO 25 (PWM)

```
// =====
// PROYECTO INTEGRADOR: AHT20 + OLED + Ventilador automático
// SDA → GPIO 21 | SCL → GPIO 22 | Fan PWM → GPIO 25
// =====

#include <Wire.h>
#include <Adafruit_AHTX0.h>
#include <Adafruit_SSD1306.h>

#define FAN_PIN 25

// Objetos
Adafruit_AHTX0 aht;
Adafruit_SSD1306 display(128, 64, &Wire, -1);

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);

  aht.begin();
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);

  ledcAttach(FAN_PIN, 25000, 8); // Fan PWM 25kHz, 8 bits
  display.clearDisplay();
}

void loop() {
  // Leer sensor
  sensors_event_t temp, hum;
  aht.getEvent(&hum, &temp);

  // Control ventilador: ON sobre 25°C, lineal hasta 40°C
  int fan_speed;
  if (temp.temperature < 25) {
    fan_speed = 0;
  } else if (temp.temperature >= 40) {
```



```

    fan_speed = 255;
} else {
    fan_speed = map((int)(temp.temperature * 10), 250, 400, 1, 255);
}
ledcWrite(FAN_PIN, fan_speed);

// Mostrar en OLED
display.clearDisplay();
display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);
display.setCursor(10, 5);
display.printf("Temp: %.1f C", temp.temperature);
display.setCursor(10, 25);
display.printf("Hum: %.1f %%rH", hum.relative_humidity);
display.setCursor(10, 45);
display.printf("Fan: %3d/255", fan_speed);
display.display();

delay(500);
}

```

✓ Checklist de depuración

1. Verificar que SDA y SCL están en los GPIOs correctos y con pull-ups
2. Ejecutar escáner I2C primero para confirmar direcciones
3. Verificar que los voltajes de alimentación son correctos (3.3V para ESP32 y sensores)
4. Si el PWM del ventilador "zumba" a bajo duty cycle, subir la frecuencia a 25kHz
5. Calibración: comparar lectura de AHT20 con un termómetro de referencia

11.7 Resumen y Referencia Rápida

Función	Código	Notas
Configurar PWM	<code>ledcAttach(pin, freq, res)</code>	Válido para ESP-Arduino 3.x
Cambiar duty	<code>ledcWrite(pin, duty)</code>	0 a $2^{\text{res}}-1$
Leer ADC	<code>analogRead(pin)</code>	12 bits (0-4095)
Configurar atenuación	<code>analogSetAttenuation(ADC_11db)</code>	Rango 0-3.3V
Iniciar I2C	<code>Wire.begin(SDA, SCL)</code>	Configurar pines si no son default
Escribir I2C	<code>Wire.beginTransmission(addr); ... endTransmission();</code>	Maestro → Esclavo

Función	Código	Notas
Leer I2C	<code>Wire.requestFrom(addr, n_bytes)</code>	Maestro recibe datos